

TRAVAUX PRATIQUES III

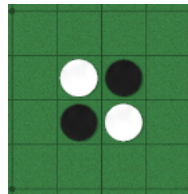
Ôtez l'eau de mon plateau!

Le but de ce TP est d'implémenter les méthodes min-max et alpha-beta sur le jeu Othello.

A Présentation : règles du jeu

Commençons par présenter le jeu Othello (source des images : <https://www.worldothello.org/about/about-othello/othello-rules/official-rules/english>). Ce jeu se joue sur un plateau de taille $n \times n$ cases où n est un entier pair tel que $n \geq 4$. Chaque joueur pose à tour de rôle un pion sur un plateau, tant qu'il peut capturer des pions adverses. Le joueur J1 pose des pions noir, et J2 pose des pions blancs. Le but est d'avoir le plus possible de pions de sa couleur sur le plateau à la fin de la partie, lorsque plus aucun joueur ne peut jouer ou que le plateau est entièrement rempli.

Le jeu démarre dans la position initiale contenant quatre pions sur les quatre cases du plateau (qui existent car n est pair), une diagonale noire et une diagonale blanche comme ci-dessous :



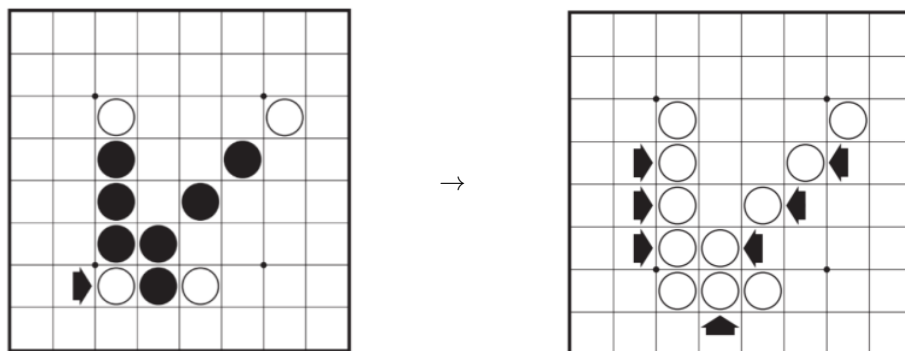
Ce sont les noirs (J1) qui jouent en premier, en posant un pion de leur couleur sur le plateau.

Tout l'intérêt du jeu est que l'on peut capturer les pions de la couleur adverses en les prenant en tenaille entre deux pions de sa couleur. Au moment où un joueur pose un pion de sa couleur, s'il encadre un alignement de pions adverses avec un autre pion de sa couleur déjà placé, tous les pions adverses ainsi pris en sandwich deviennent des pions de sa couleur.

Schématiquement, si le joueur blanc vient de placer le pion A ou B dans la situation suivante, il capture les trois pions noirs (qui deviennent des pions blancs) :



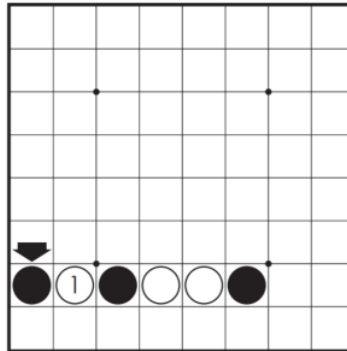
Chaque encadrement formé donne une capture, que ce soit en ligne, en colonne ou en diagonale. Donnons un exemple sur un plateau de taille 8×8 . À gauche, la flèche montre le pion blanc qui vient d'être placé et forme 3 encadrements. À droite, les flèches montrent tous les pions qui ont été capturés.



Toutefois, seuls les pions qui viennent d'être encadrés par le nouveau pion posé sont capturés, et pas les éventuels autres pions encadrés si d'autres encadrements sont créés par les captures. De plus, si une ligne présente une alternance

de pions des deux couleurs, poser un pion au bout de la ligne ne retourne qu'une ligne continue de pions adverses (sans sauter par dessus un autre pion de sa propre couleur).

Par exemple, dans la figure suivante, seul un pion blanc est retourné par le coup indiqué par la flèche :



Rappelons comment le gagnant est évalué : lorsque plus aucun joueur ne peut jouer (soit car le plateau est plein, soit car personne ne peut effectuer de capture), le joueur ayant le plus de pions de sa couleur l'emporte.

Si un joueur ne peut pas jouer alors que c'est à lui de jouer, **il passe son tour**. Sinon, un joueur ne peut pas choisir de passer son tour. Il n'y aura donc pas de partie infinie.

B Modélisation

B.1 Représentation OCaml

On utilise les types suivants pour représenter un état :

```
1 (** Type des cases du plateau *)
2 type couleur = Vide | Noir | Blanc
3
4 (** Type des configurations *)
5 type etat = { joueur : couleur; plateau : couleur array array }
```

Un plateau $n \times n$ y est représenté par une matrice $n \times n$ (un tableau de tableaux). Les cases sont indexées comme dans une matrice, avec la case en haut à gauche d'indice $(0).(0)$.

La plupart des fonctions permettant de jouer sont déjà précodées. Vous sont fournis par mail :

- `othello.mli` qui contient l'interface des fonctions précodées.
- `othello.cmi` qui est une version compilée de cette interface ¹.
- `othello.cmo` qui est le fichier objet compilé du code source. Vous n'avez pas le code source !
- `tp0thello.ml` où vous devrez coder.
- `Makefile` qui sert à compiler (cf suite).

Un fichier `Makefile` est un fichier contenant des instructions de compilation. Pour le lancer, on utilise la commande terminal `make`. Ce `Makefile`-ci est configuré pour qu'en outre :

- `make build` compile sans exécuter
- `make run` lance l'exécutable sans compiler.
- `make`, ou `make all` compile puis exécute.

1. Lisez `othello.mli` pour voir les fonctions auxquelles vous avez accès.

B.2 Fonctions intermédiaires utiles

Dans cette partie, on code le minmax sans heuristique ni élagage, mais avec mémorisation !

2. Écrire une fonction `compte_couleur` qui prend en argument un état et renvoie une paire `n,b` où `n` est le nombre de pions Noirs dans l'état et `b` le nombre de Blancs.

1. En OCaml, les interfaces se compilent.

3. Écrire une fonction `eval : etat -> int` qui évalue un état en lui associant la différence entre le nombre de Noirs et de Blancs.
 ⚠ Pour l'instant, on évalue uniquement les noeuds terminaux et on doit normalement donner un score $+\infty$, $-\infty$ ou 0 (cf cours). Ici, on peut vouloir garder des scores à points pour savoir de combien un joueur gagne. On compte le nombre de pions noirs moins le nombre de pions blancs. J1 (Noir) gagne si ce score est strictement positif, J2 gagne si le score est strictement négatif, et c'est un nul sinon.
4. Écrire une fonction `gagnant : etat -> couleur` qui prend en argument un état terminal d'une partie et renvoie le gagnant de la partie. Si la partie est nulle, on renvoie `Vide`. On ne se préoccupera pas de savoir si l'état donné en argument est terminal.

C Minmax

C.1 Minmax vanilla

5. Écrire une fonction `minmax : etat -> int` qui calcule le score d'une position et la renvoie. Exceptionnellement, cette fonction devra utiliser la fonction `eval` sur les positions terminales pour permettre le calcul du score plutôt que du simple gagnant.
 Ce minmax n'est pas à profondeur bornée, mais doit être mémorisé.
6. Que dire de l'Othello 4×4 ? Un joueur a-t-il une stratégie gagnante? Si oui, lequel et de combien gagne-t-il?

On sait désormais quel joueur a une stratégie gagnante, mais on préférerait connaître cette stratégie gagnante! On veut maintenant récupérer cette stratégie, que l'on va représenter sous forme d'un tableau associatif. Une association (clé, valeur) de cette table est un couple de deux états (s, s') tel que depuis la position s , la stratégie f indique de jouer le coup menant à s' , i.e. si $s' = f(s)$.

Rappel : Pour adapter l'algorithme minmax afin qu'il renvoie une stratégie optimale, il ne faut pas se contenter de calculer les scores des noeuds mais aussi de retenir pour chaque noeud non terminal contrôlé par J1 quel enfant a donné le score maximal (pour une stratégie pour J1) ou pour chaque noeud non terminal contrôlé par J2 quel enfant a donné le score minimal (pour une stratégie pour J2).

À vous de jouer maintenant, vous devez adapter votre algorithme minmax précédent pour qu'il calcule non seulement l'extremum des enfants mais aussi quel enfant a donné l'extremum.

7. Écrire une fonction `dict_minmax` telle que `dict_minmax s0 j` calcule un tableau associatif représentant une stratégie optimale pour le joueur j depuis la position $s0$.
Indication : n'hésitez pas à faire une/des fonctions intermédiaires, et/ou à développer certains morceaux faits par des fonctions précédées. List.map ne sera plus nécessairement votre ami ici...

Remarque : On pourrait stocker dans le même dictionnaire les stratégies optimales de J1 et de J2 (qui maximise / minimise le score), puisque leurs coups portent sur des ensembles disjoints d'états. Faites le si vous le souhaitez.

C.2 Minmax avec heuristiques

8. Écrire une fonction `heur_naive` qui pour évaluer une position se contente de la différence entre Noirs et Blancs.
 Cette fois, il est bien plus important de distinguer l'évaluation d'un état terminal (un des joueurs **gagne**) de celle d'un état estimé avec cette heuristique peu fiable. On fera donc bien attention de donner les scores $+\infty$, $-\infty$ ou 0 aux états terminaux, indépendamment du nombre de pions que le joueur gagnant possède en plus de l'autre.
9. En jouant un peu, on se rend compte que les bords et a fortiori les coins sont des cases intéressantes car plus difficiles pour l'adversaire à retourner (les coins sont même impossibles à retourner!). Au contraire, les cases adjacentes à ces positions sont plutôt faibles.
 Écrire une fonction `heur_points` qui évalue une position en pondérant plus fort les bords et les coins.
 La valeur exacte sera plus tard laissée à votre interprétation. Pour le testeur, vous devez donner 20 points aux coins, 3 points aux pions des bords et 1 point aux autres. On donnera toujours un score infini au gagnant d'une position terminale.

Nous avons quelques heuristiques à utiliser et comparer, il faut maintenant borner le calcul de minmax à une certaine profondeur fixée, comme décrit dans le cours.

10. Écrire une fonction `minmax_heur`, telle que `minmax_heur h pmax s0` calcule la valeur de la position `s0` via minmax à profondeur maximale `pmax` et en utilisant l'heuristique `heur` pour évaluer les positions.

Encore une fois, c'est bien de savoir qui semble avoir l'avantage, mais c'est mieux de savoir comment bien jouer! On cherche une stratégie à suivre, pas seulement une valeur pour la position sur laquelle on est.

On va changer un peu de méthode. Contrairement à tout à l'heure, nous ne calculons pas tout l'arbre en une seule fois, mais seulement à p coups après notre position. Donc, au coup suivant, on pourra calculer plus loin et changer notre stratégie. On veut à partir de maintenant **recalculer la stratégie à suivre à chaque coup**. Il est donc inutile de savoir quels coups il faudra jouer plus tard, puisqu'on les recalculera. On va se contenter de calculer uniquement le premier coup à jouer, c'est à dire quel enfant maximise ou minimise le score sous la racine (J1 ou J2 à la racine).

11. Écrire une fonction `strat_minmax_heur`, telle que `strat_minmax_heur h pmax s0` renvoie le coup optimal à jouer pour le joueur contrôlant la position `s0`. Dans le cas d'une position terminale, la fonction soulèvera l'exception `Not_found`. (Cette exception n'est pas à définir, elle existe dans le module `Hashtbl`.)

Il "suffit" d'adapter votre minmax précédent pour retenir quel coup a donné l'extremum, en plus de l'extremum. Je vous conseille de ne plus utiliser la fonction `extremum_list` donnée en cours mais de faire votre propre fonction, plus à même de répondre à ce problème précis.

12. Si vous avez des idées, testez de nouvelles heuristiques! Amusez-vous!

Par exemple, pour tenter de raffiner un peu votre heuristique précédente, vous pouvez ensuite choisir de dire qu'un pion vaut par défaut 2 points, et ajouter +8 à sa valeur s'il est sur un coin, +2 s'il est sur un bord, -2 s'il est adjacent à un coin, et -1 s'il est adjacent à un bord (sauf si c'est lui-même un bord ou un coin). Il est toujours délicat de trouver une bonne heuristique².

Rappel : une fonction `simule` est fournie si vous voulez faire s'affronter vos différentes fonctions/heuristiques. La fonction partielle `strat_minmax_heur h pmax` codée précédemment est exactement une stratégie (fonction qui à un état `s0` associe l'état donné par le coup à jouer).

Pour vos fonctions précédentes, on donne aussi une fonction `strat` qui transforme une table de hachage en vraie stratégie (fonction).

N'hésitez pas à faire un tournoi d'Othello entre toutes vos stratégies, si vous avez de l'avance;).

D Alpha-Beta

13. Écrire une fonction `alpha_beta` telle que `alpha_beta heuri prof_max s0` calcule le score de la position `s0` obtenu par algorithme minmax avec élagage $\alpha - \beta$ non mémoisé, en utilisant l'heuristique `heuri` appliquée à la profondeur `prof_max`.
14. Adaptez votre fonction précédente pour écrire une fonction `strat_alpha_beta` qui renvoie le coup optimal à jouer pour le joueur contrôlant la position `s0`, obtenu par algorithme minmax avec élagage $\alpha - \beta$.

Aide : On pourra par exemple utiliser des variables globales retenant le meilleur enfant vu jusqu'à maintenant, ainsi que sa valeur. Seuls les enfants de la racine (à profondeur 0) nous intéressent (ne mettez pas à jour en comparant avec les valeurs de petits enfants ou des descendants...).

⚠ il faudra faire bien attention au point suivant :

- Il peut y avoir une égalité entre les enfants donnant le meilleur score (maximal ou minimal selon le joueur). Dans ce cas, **les enfants ne se valent pas!** En effet, on peut avoir élagué les enfants d'un des enfants candidat possible sur une égalité à α ou à β , et celui-ci pourrait donc en réalité être pire que son frère gauche/droit qui semble avoir le même score. Il faut donc sélectionner le **premier** enfant trouvé parmi ceux qui donnent le meilleur score.
 - Réfléchissez bien à comment vous allez initialiser vos variables globales. Pensez peut-être à traiter à part le cas d'un noeud qui n'a pas d'enfant.
15. Faites affronter vos algorithmes $\alpha - \beta$ entre eux et avec vos minmax non élagués. Vérifiez que les stratégies obtenues sont bien **les mêmes** à heuristique fixée. (Sinon, vous avez un bug! Tout ce qui doit changer c'est le temps d'exécution de vos programmes.)

E Pour aller plus loin

16. Coder toutes les fonctions décrites dans l'interface `othello.mli`.

2. Pour l'othello, des études ont déjà été faites qui semblent indiquer qu'une heuristique de ce genre a peu de chance d'être bonne... il vaut mieux considérer le nombre de coups rendus possibles, par exemple. Mais ces heuristiques-ci sont les plus simples à coder, et on s'en contentera pour un TP!